

Object-Oriented Languages

Programming Languages Intro

Dr. Igor Steinmacher

Data Abstraction

- Extend imperative notion of type by:
 - Providing *encapsulation* of data/functions
 - Example: stack of int's
 - Separation of interface from implementation
 - Abstract Data Types! (ADT)
- **Encapsulation**
 - Logically related constructs to be grouped into a new entity

A simple queue in C

```
struct Queue {  
    void **array;  
    int head;  
    int tail;  
    int size;  
};
```

```
void init( struct Queue *, int );
```

```
bool is_empty( struct Queue * );
```

```
int size( struct Queue * );
```

```
void *front( struct Queue * );
```

```
void enqueue( struct Queue *, void * );
```

```
void dequeue( struct Queue * );
```

Why Data Abstraction?

- Encapsulate
 - Data + Functions
- As a module so one can use
 - Functions via public interface
 - Data type definitions

Object Model is Born

- ADT evolved!
 - Encapsulation with hiding of information
 - Supports the inheritance of certain functions
 - Dynamic method selection
- “**Objects** as ‘capsules’ containing both data and the operations that manipulate it and which provides an interface for the outside world through which the object is accessible”

Queue... in Java

```
public class Queue {
    private static MAX_SIZE=100;
    private int size;
    private int items[];
    private int tail;
    private int head;

    public Queue() {
        head = -1;
        tail = -1;
        this.items = new int[MAX_SIZE];
    }

    boolean isFull() {
        if (head == 0 && tail == SIZE - 1)
            return true;
        return false;
    }

    public boolean isEmpty() {
        if (head == -1)
            return true;
        return false;
    }

    public int size() {
        return size;
    }
}
```

```
    public void enqueue(int element) {
        if (head == -1)
            head = 0;
        tail++;
        items[tail] = element;
        System.out.println("Inserted " + element);
    }

    public int dequeue() {
        int element;
        if (isEmpty()) {
            System.out.println("Queue is empty");
            return (-1);
        } else {
            element = items[head];
            if (head >= tail) {
                head = -1;
                tail = -1;
            }
            else {
                head++;
            }
            System.out.println("Deleted -> " + element);
            return (element);
        }
    }
}
```

Classes

- Class is a model for a set of objects
 - Names and types for data (attributes)
 - Function signature (methods)
 - Function and variables visibility (encapsulation)
 - **Private** are not accessible from the outside
 - **Public** are visible to all

```
class Counter{  
    private int x;  
    public void reset() {  
        x = 0;  
    }  
    public int get() {  
        return x;  
    }  
    public void inc() {  
        x = x+1;  
    }  
}
```

Inheritance

- Class inherits attributes and methods from a superclass
- Visibility needs to be respected
 - In Java, private cannot be accessed outside

```
class Counter{  
    private int x;  
    public void reset(){  
        x = 0;  
    }  
    public int get(){  
        return x;  
    }  
    public void inc(){  
        x = x+1;  
    }  
}
```

```
class NewCounter extends Counter{  
    private int num_reset = 0  
    public void reset(){  
        super.reset();  
        num_reset++;  
    }  
    public int how_many_resets(){  
        return num_reset;  
    }  
}
```


Inheritance

- Everything `Counter` understands, `NewCounter` also does
- `NewCounter` re-uses the methods `get ( )` and `inc ( )`:
 - Implementation is inherited from the superclass
- It redefines the method `reset ( )`
 - Method **overriding**
- It also defines a new method `howManyResets ( )`

Dynamic Method Lookup

- A method can be redefined in objects belonging to other classes (as we saw in our previous slide)
 - there can be many versions of a method
- We have to look at the actual type of the object
 - this is also **dynamic dispatch**

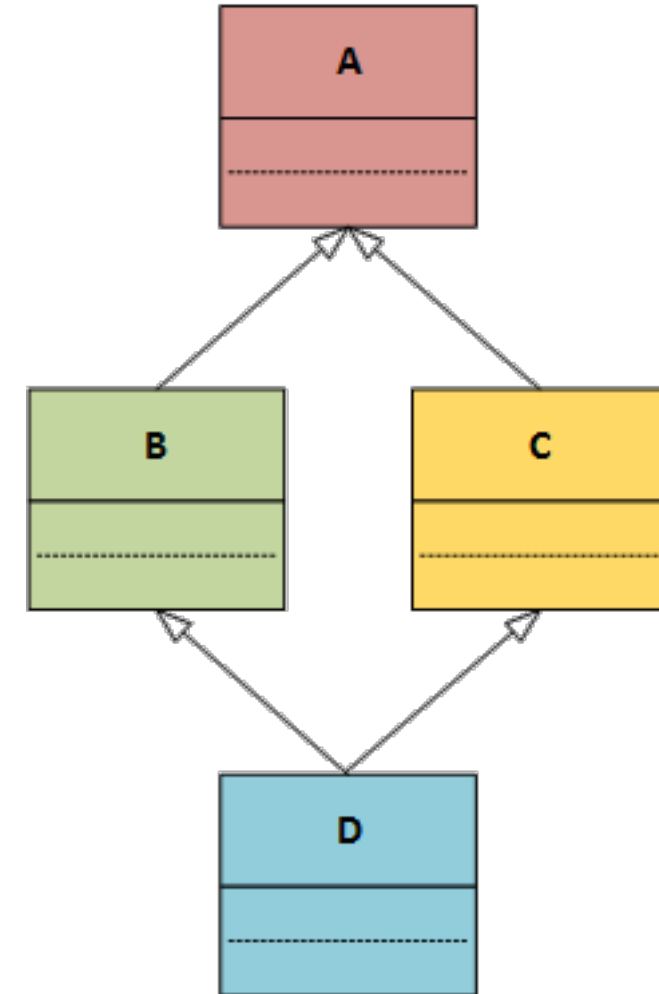
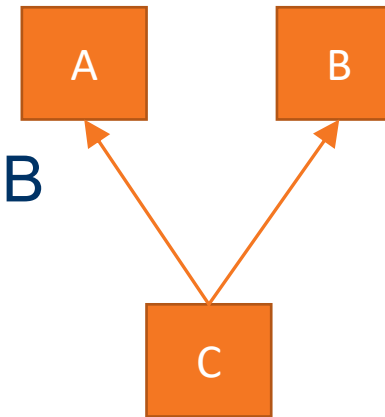
```
Counter counters[] = new Counter[10];  
counters[0] = new Counter();  
counters[1] = new NewCounter();  
...  
for (int i = 0; i < 10; i++){  
    counters[i].reset();  
}
```

Multiple Inheritance

- Allows a class to be a subclass of more than 1 class
- Facilitates code reuse
 - A Teaching Assistant has attributes from Student and Teacher
- But...more complicated semantics
- C++, Eiffel, OCaml

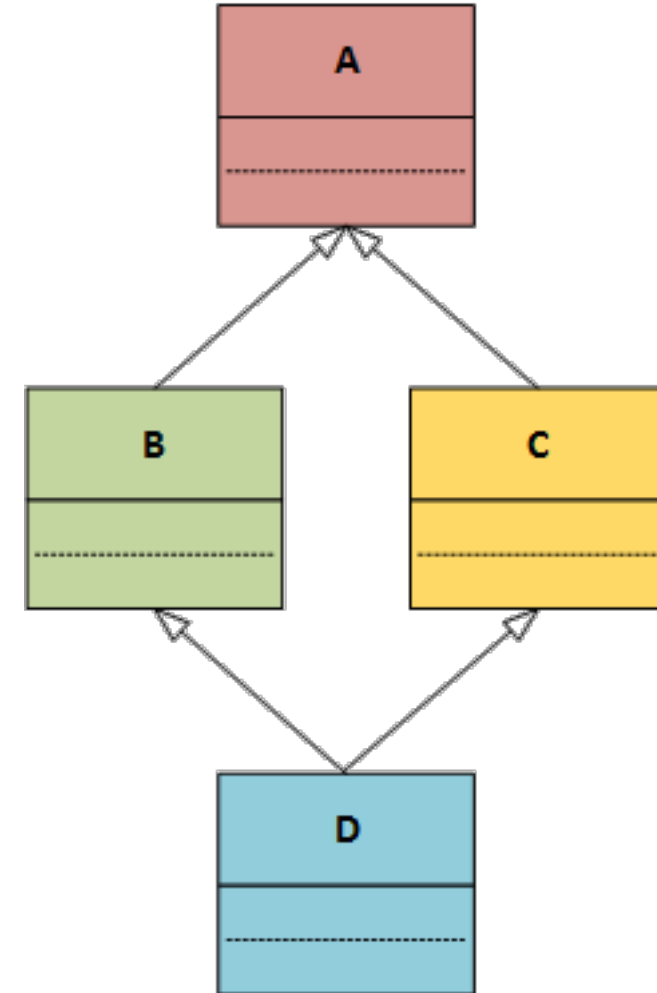
Multiple Inheritance: Conflicting Names, Diamond problem

- Class C inherits from Class A and Class B
 - A and B have methods called sum()
 - Will C inherit A.sum() or B.sum()?
 - A declares a variable x which is an integer; B declares x as a string
 - How many x's will C have? If one, what type?
- Diamond problem (see the figure)
 - Same problem discussed
 - B and C will have all attributes and methods from A



Multiple Inheritance: Conflicting Names, Diamond problem

- Eiffel solution: **rename**
 - D redefines the inherited methods: e.g. sumB() for B.sum(), and sumC() for C.sum()
 - xA for A.x and xB for B.x
- Ocaml
 - Methods (and attributes) are inherited in the **same order**, with newly inherited method overriding any existing methods
- C++
 - **Virtual** inheritance
 - only one copy of a base class attributes and methods are inherited by grandchild
- Java
 - **Does not allow** multiple inheritance



Other topics: Reading

- Subtypes
 - Interfaces and Abstract classes in Java
- Templates
- Generics
- Aspect Orientation

**See you in our next
Lesson!**